



AI201 | Applications of AI Engineering

Applications of AI Engineering Summer 2026 (@ Section 2A | Thursday 5PM - 7PM PT)

Personal Member ID#: **71525**

Need help? Post on our [class slack channel](#) or email us at support@codepath.org

[Welcome](#)[Overview](#)[Tinker \(Lab\)](#)[Show \(Project\)](#)[Report](#)[Getting Started](#)[► Course Info](#)[AI Prompting
Guide](#)[Course Progress](#)[▼ Module 1](#)[Week 1](#)[Week 2](#)[Week 3](#)[Week 4](#)[► Module 2](#)[► Module 3](#)

Reminder: Project 2 is due by **Sunday, June 14th at 11:59PM PDT.**

[Submit](#)

Show What You Know: FitFindr



~8–9 hours total

Thrifting is not just a way to shop — it's a whole process. You're searching across multiple platforms, trying to picture how something fits with what you already own, figuring out if the price is actually good, and ultimately deciding whether it's worth it. That process involves a lot of searching, comparing, and reasoning that could be significantly more useful with the right tools.

In this project, you'll build **FitFindr**: a multi-tool AI agent that helps users find secondhand pieces and figure out how to wear them. The agent orchestrates a set of tools in response to a natural language request — searching listings, evaluating fit against an existing wardrobe, and generating a shareable outfit description — while handling the messy reality of what happens when a tool fails or returns nothing useful.

Last week you built a retrieval system. This week you're building something that acts: an agent that decides which tools to call, in what order, and what to do when things don't go as planned.

Goals

By completing this project, you will be able to:

- Design a set of tools with clearly defined interfaces, inputs, and outputs.
- Implement a planning loop that determines which tools to call and when.
- Manage state across multiple tool calls within a single session.
- Build error handling for each tool that keeps the agent useful when something breaks.
- Document agent behavior so someone reading your README understands what the agent is doing, not just that it's working.

Features

Required Features

- ☐ **Three or more tools with defined interfaces:** Build at least 3 distinct tools. Each tool must have a clearly defined function signature: what it accepts as input and what it returns. The following three are required; you may add more.
 - ☐ `search_listings(description, size, max_price)` — Searches the mock listings dataset and returns matching items. Must handle the case where no matches are found.
 - ☐ `suggest_outfit(new_item, wardrobe)` — Given a specific item and the user's current wardrobe, suggests one or more complete outfit combinations. Must handle an empty or minimal wardrobe.
 - ☐ `create_fit_card(outfit, new_item)` — Generates a short, shareable description of a complete outfit — the kind of thing someone would caption an Instagram post with. Must produce something different each time for different inputs.
- ☐ **Planning loop:** Your agent must use a loop (or equivalent reasoning mechanism) that selects which tools to call based on what's been returned so far. The agent should not call all tools in a fixed sequence regardless of context — it should respond to what it receives. Document how your planning loop works in your README.

- ☐ **State management across tool calls:** Information returned by one tool must be available to subsequent tools in the same session. For example, the item found by `search_listings` should flow into `suggest_outfit` without the user having to re-enter it. Show this in your demo video.
- ☐ **Error handling for each tool:** Every tool must handle its own failure mode. "Fail silently" or "crash the agent" are not acceptable error handling strategies. At minimum: if a tool returns an empty result or encounters an error, the agent should communicate this to the user and either try a fallback strategy or ask for more information. Document your error handling strategy in your README.
- ☐ **Multi-step workflow:** Your demo must include at least one complete multi-step interaction that uses all 3 required tools in sequence — from initial query to final fit card. Show the full flow.

Stretch Features

Complete any of these for extra credit. Update your `planning.md` before starting each one.

- ☐ **Price comparison tool:** Add a fourth tool that, given an item, estimates whether the price is fair based on comparable listings in the dataset.
- ☐ **Style profile memory:** Allow the agent to remember a user's style preferences across sessions, so they don't have to re-describe their wardrobe every time.
- ☐ **Trend awareness:** Add a tool that checks recent posts or tags on a public fashion platform to surface what styles are currently popular in the user's size range.
- ☐ **Retry logic with fallback:** If `search_listings` returns no results, automatically retry with loosened constraints (e.g., remove size filter) and inform the user what was adjusted.

Hints

- Design your tools on paper before implementing them. What does each one actually need? What can it reasonably return? Tools with vague inputs and outputs create hard-to-debug agents.
- Test tools in isolation first. An agent that uses three broken tools is three times harder to debug than one broken tool.
- Your planning loop doesn't have to be complex to be real. What matters is that the agent's behavior changes based on what was returned — not that the loop is architecturally impressive.
- Error handling is a design decision before it's a coding problem. For each tool, ask: what would be least frustrating for the user if this fails? Then implement that.

- The fit card should actually sound like something worth sharing. If it reads like a product description, that's a signal to revisit the prompt.

Tools and Setup

This project uses the same free LLM from Project 1 — no new accounts or credits required.

Recommended stack

Component	Tool	Notes
LLM	Groq (<code>llama-3.3-70b-versatile</code>)	Free tier — same account as Project 1
Mock data	Provided in starter repo	No external API needed

Getting started

- ☐ **Fork** the [FitFindr starter repo](#), then **clone your fork** locally.
- ☐ **Create and activate a virtual environment** from inside your cloned repo:

```
python -m venv .venv
source .venv/bin/activate           # Mac/Linux
source .venv/Scripts/activate      # Windows (Git Bash)
# or: .venv\Scripts\activate       # Windows (Command Prompt)
```

You should see `(.venv)` in your terminal prompt.

- ☐ **Install dependencies:**

```
pip install -r requirements.txt
```

- ☐ **Create a `.env` file** in your repo root. This file is already listed in `.gitignore` — never commit it.

```
GROQ_API_KEY=your_key_here
```

This is the same key from Project 1. If you haven't signed up yet, get a free key at [console.groq.com](#) — no credit card required.

Milestone 1: Explore the Starter Repo and Understand the Problem

 ~30 min

Before designing anything, understand what you're building end-to-end. Then get oriented in the data — your tool designs will only be as good as your understanding of the inputs you're working with.

Here's what a complete FitFindr interaction looks like:

User: "I'm looking for a vintage graphic tee under \$30, size M. I mostly wear baggy jeans and chunky sneakers."

Step 1 — Search: `search_listings("vintage graphic tee", size="M", max_price=30.0)` returns 3 matching listings sorted by relevance. FitFindr picks the top result: *"Faded Band Tee — \$22, Depop, Good condition."*

Step 2 — Suggest outfit: `suggest_outfit(new_item=<band tee>, wardrobe=<user's wardrobe>)` returns: *"Pair this with your wide-leg jeans and platform Docs for a classic 90s grunge look. Roll the sleeves once and tuck the front corner slightly for shape."*

Step 3 — Fit card: `create_fit_card(outfit=<suggestion>, new_item=<band tee>)` returns: *"thrifted this faded band tee off depop for \$22 and honestly it was made for my wide-legs ❤️ full look in my stories"*

Error path: If `search_listings` returns nothing, FitFindr tells the user what to try differently and stops — it does not call `suggest_outfit` with empty input.

- ☐ Open `data/listings.json` and read through 5–10 listings. Note the available fields: `id`, `title`, `description`, `category`, `style_tags`, `size`, `condition`, `price`, `colors`, `brand`, `platform`. These are the fields your `search_listings` tool will filter against.
- ☐ Open `data/wardrobe_schema.json` and read the wardrobe structure. Note the fields available on each wardrobe item. This is the schema your `suggest_outfit` tool will receive as input.
- ☐ Read `utils/data_loader.py`. It provides `load_listings()`, `get_example_wardrobe()`, and `get_empty_wardrobe()` — helper functions you should use rather than rewriting.

- ☐ In your own words, write a 2–3 sentence description of what FitFindr needs to do. Include what triggers each tool and what happens when something fails. Add this to the **A Complete Interaction** section of `planning.md` — that section has an example query waiting for you to trace through.

Checkpoint

You can describe the listings data structure from memory (key fields and their types). You understand the difference between `get_example_wardrobe()` and `get_empty_wardrobe()` and when you'd use each in testing.

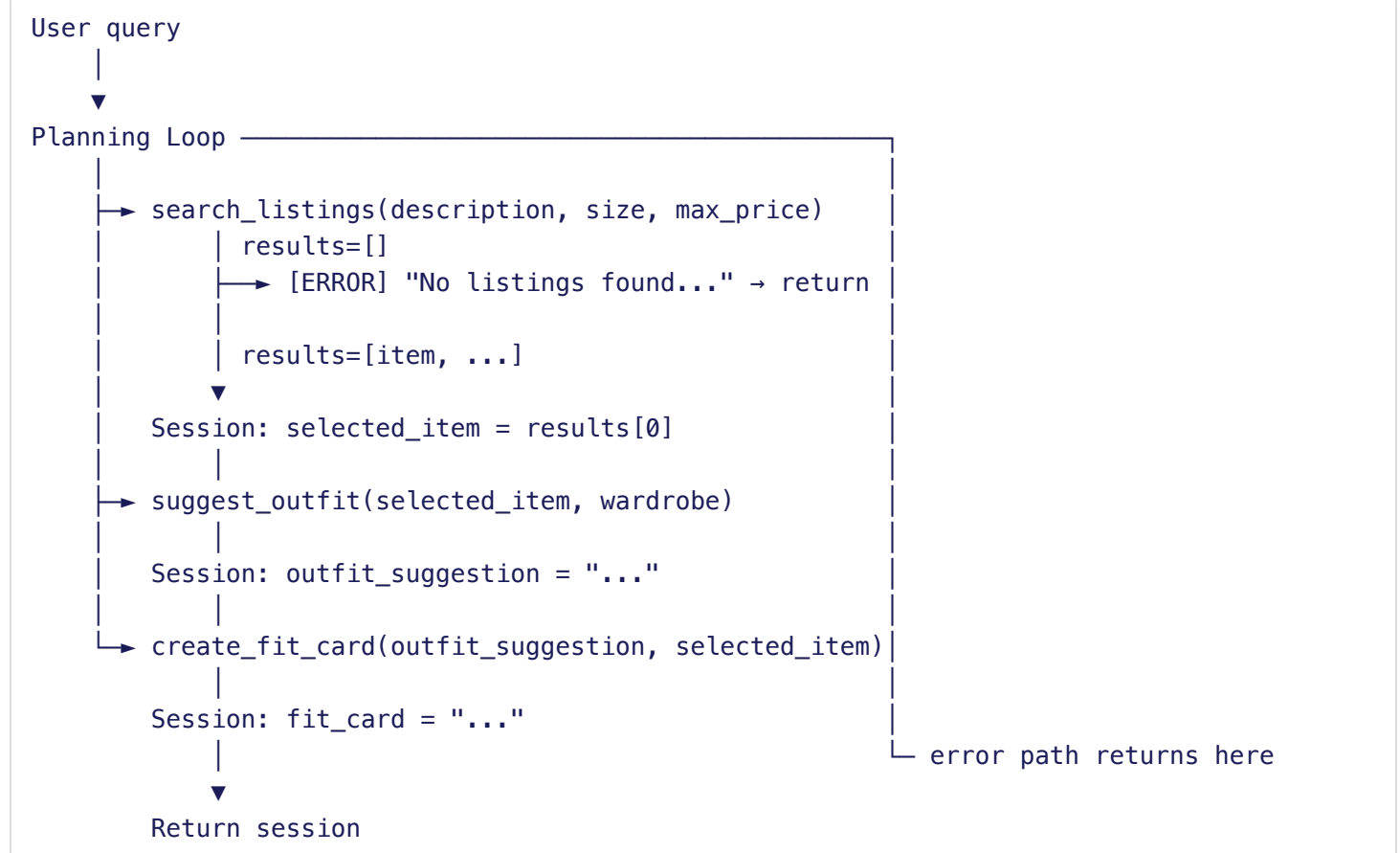
Milestone 2: Write Your Spec Before Any Code

~1 hour

The section headers in `planning.md` are pre-populated — your job is to fill in each one with specific, implementation-ready content. Unlike Project 1, there's no further scaffolding inside each section. Designing an agent starts with deciding what it should actually do, not filling in blanks. A vague planning loop description ("it decides what to do next") will produce a vague implementation — and a vague prompt to an AI tool.

- ☐ Open the `planning.md` already in your cloned repo. Fill in each section with specific, implementation-ready content.
- ☐ For each of the three required tools, fill in: what it does, its exact input parameters (name, type, meaning), what it returns, and what the agent does if it fails or returns nothing. "Returns a list" is not specific enough — describe what's in the list.
- ☐ For the planning loop, describe the actual conditional logic — not just "it decides what to call next" but the specific branches. For example: *"After search_listings runs, check if results is empty. If yes, set an error message in the session and return early. If no, set selected_item = results[0] and proceed to suggest_outfit."* Your description should be specific enough that someone else could implement it from your words alone.

- ☐ Draw an agent diagram and add it to the `## Architecture` section of `planning.md`. A good diagram includes: labeled nodes for each component (user, planning loop, each tool, session state), labeled arrows showing what data flows between them, and a visible error branch where the flow terminates early. Here's a minimal example to build from:



You'll share this diagram with an AI tool when implementing the planning loop — a clear diagram produces more accurate generated code than a text description alone.

Format requirement: Your diagram must be **text-based** — ASCII art (like the example above) or a [Mermaid](#) diagram in a ````mermaid` fenced block — committed directly in `planning.md`. An embedded image or screenshot is **not** accepted, because it can't be read by the automated grader.

- ☐ Fill in the `## AI Tool Plan` section. For each milestone, describe: which AI tool you'll use (Claude, Copilot, ChatGPT — any is fine), what you'll give it as input (name the specific planning.md sections and diagram), what you expect it to produce, and how you'll verify the output matches your spec before using it. "I'll use AI to help me code" is not a plan. Here's what a plan looks like:

"For search_listings, I'll give Claude the Tool 1 block from planning.md (inputs, return value, failure mode) and ask it to implement the function using load_listings() from the data loader. Before running it, I'll check that the generated code filters by all three parameters and handles the empty-results case. Then I'll test it with 3 queries."

- ☐ For the complete interaction walkthrough at the bottom of `planning.md`, trace through the example query step-by-step: which tool is called first, with what exact inputs, what does it return, what gets called next, and what does the user see at the end.
- ☐ Review each error handling row in the table. Each response should be specific and actionable — not just "tell the user there was an error" but what the agent actually says and what (if anything) it offers instead.

Checkpoint

`planning.md` describes all three tools with specific inputs and return values. The planning loop section describes conditional logic, not just intent. The agent diagram shows how data and control flow between the planning loop, tools, and state. The AI Tool Plan names specific spec sections you'll use as prompts. The complete interaction walkthrough traces a full example query step-by-step through all three tool calls.

Milestone 3: Build and Test Each Tool in Isolation

~2–3 hours

Implement each tool individually and test it with hardcoded inputs before connecting anything. This is the most important discipline in agent development — an agent with three untested tools is three times harder to debug

than three individually-tested tools wired together. Do not move to Milestone 4 until each tool works correctly on its own, including its failure mode.

- ☐ All three tool stubs are already in `tools.py` in your starter repo. Open it and read through the docstrings — each function's Args, Returns, and TODO steps are pre-written. Implement the functions in `tools.py` directly; do not create separate files for each tool.
- ☐ Use your `planning.md` tool specs to prompt an AI tool to implement each function. Give it one tool at a time — paste in that tool's spec block (what it does, inputs, return value, failure mode) and ask it to implement that function in `tools.py`. Review each generated function against your spec before running it: does it match the parameters you defined? Does it handle the failure mode you described?
- ☐ Implement `search_listings(description, size, max_price)`. Use `load_listings()` from `utils/data_loader.py` — don't re-implement the file loading. `create_fit_card` takes both `outfit` and `new_item` as arguments (check the stub signature).
- ☐ Implement `suggest_outfit(new_item, wardrobe)`. This tool calls the LLM — use Groq's `llama-3.3-70b-versatile` with your `GROQ_API_KEY` from `.env`. Handle the empty wardrobe case — the agent should not crash if `wardrobe['items']` is empty.
- ☐ Implement `create_fit_card(outfit, new_item)`. This also calls the LLM. Guard against an empty `outfit` string — return an error message string rather than crashing. Run it several times on the same input and verify the outputs vary; if they're identical, increase the LLM temperature.

- ☐ Write pytest tests for each tool in a `tests/` folder. Create a `tests/test_tools.py` file and write at least one test per failure mode:

```
# tests/test_tools.py
from tools import search_listings

def test_search_returns_results():
    results = search_listings("vintage graphic tee", size=None, max_price=50)
    assert isinstance(results, list)
    assert len(results) > 0

def test_search_empty_results():
    results = search_listings("designer ballgown", size="XXS", max_price=5)
    assert results == [] # empty list, no exception

def test_search_price_filter():
    results = search_listings("jacket", size=None, max_price=10)
    assert all(item["price"] <= 10 for item in results)
```

Run with `pytest tests/` and confirm all tests pass before moving to Milestone 4.

💡 **Why pytest?** Hardcoding test calls inside functions makes them easy to forget and hard to re-run. pytest lets you run all your tool tests with one command and immediately see which ones fail — especially useful when you change something in the planning loop later.

📌 Checkpoint

You can call each of the three tools directly with test inputs and get sensible output. Each failure mode — no listings found, empty wardrobe, incomplete outfit input — returns a specific, informative message rather than raising an exception or returning nothing.

Milestone 4: Wire Up the Planning Loop and State

~2 hours

Connect your tools through a planning loop and test that state passes correctly between them. The key test at this milestone is whether your agent responds differently to different inputs — not just runs the same sequence every time. If your agent calls all three tools unconditionally regardless of what `search_listings` returns, the planning loop isn't working yet.

- ☐ Open `agent.py` in your starter repo. The session dict structure and `run_agent()` signature are already in place — read through the TODO steps and match them against the Planning Loop section of your `planning.md`. Then implement `run_agent()` following the numbered steps in the file.
- ☐ Use your agent diagram and Planning Loop + State Management sections from `planning.md` to prompt an AI tool to help you implement the planning loop. Share the full diagram and both spec sections. When you receive generated code, review it before running: does it branch on the `search_listings` result? Does it store values in the session dict? Does it not call all three tools unconditionally? Revise anything that doesn't match your spec.
- ☐ Implement `handle_query()` in `app.py`. The Gradio layout is already wired — your job is to call `run_agent()` and map the session dict to the three output panel strings. The TODO in `handle_query()` has numbered steps to follow.
- ☐ Run a complete interaction using the example query from your `planning.md` walkthrough. Verify state is passing correctly: print `session["selected_item"]` and confirm it's the exact same dict passed into `suggest_outfit`. Print `session["outfit_suggestion"]` and confirm it's what went into `create_fit_card`. If the agent is re-prompting the user or using hardcoded values between steps, state isn't flowing.
- ☐ Test the branch path: run `python agent.py` and check that the no-results path (the second test case already in the file) returns an error message in `session["error"]` and leaves `session["fit_card"]` as `None`. The agent must not call `suggest_outfit` when `search_listings` returns an empty list.

Checkpoint

A complete query flows through all three tools with state visibly passing between them — no re-entry, no hardcoded values. The agent's behavior differs when `search_listings` returns no results vs. when it returns matches. You can point to the state object at the end of an interaction and show what it contains.

Milestone 5: Test Every Failure Mode Deliberately

 ~1 hour

Deliberately break each tool and verify the agent recovers gracefully. Don't assume your error handling works until you've actually triggered each failure. This milestone is about testing the things you hope don't happen — which is exactly when production systems fail.

- ☐ Trigger `search_listings` returning zero results. Run this directly from the terminal:

```
python -c "from tools import search_listings; print(search_listings('designer ballgown', si
```

Confirm it returns an empty list `[]` without raising an exception. Then run the full agent with the same impossible query and confirm the agent's response tells the user what failed and what they can try — not just "no results found."

- ☐ Trigger `suggest_outfit` with an empty wardrobe:

```
python -c "  
from tools import search_listings, suggest_outfit  
from utils.data_loader import get_example_wardrobe, get_empty_wardrobe  
results = search_listings('vintage graphic tee', size=None, max_price=50)  
print(suggest_outfit(results[0], get_empty_wardrobe()))  
"
```

Confirm it returns a useful string (general styling advice) rather than raising an exception or returning an empty string.

- ☐ Trigger `create_fit_card` with an empty outfit string:

```
python -c "  
from tools import search_listings, create_fit_card  
results = search_listings('vintage graphic tee', size=None, max_price=50)  
print(create_fit_card('', results[0]))  
"
```

Confirm it returns a descriptive error message string — not a Python exception.

- ☐ Screenshot or record at least one of these triggered failures — you'll show it in your demo video.

Checkpoint

All three failure modes can be triggered deliberately and produce a specific, informative agent response. You have at least one triggered failure documented (screenshot or recording) to include in your demo.

Milestone 6: Document and Record

~1–1.5 hours

Write your README and record your demo. Your README should explain how the planning loop works — not just list the tools. Someone else reading your README should understand what decisions the agent makes, not just what it does when everything goes right.

- ☐ `app.py` is already in your starter repo with the Gradio layout pre-built. If you haven't implemented `handle_query()` yet (it was part of Milestone 4), do that now — the TODO inside the function has numbered steps to follow.

- ☐ Run the app and test it end-to-end:

```
python app.py
```

Open the URL shown in your terminal (check the output — it may not always be `localhost:7860`). Make sure all three output panels populate correctly for a happy-path query before moving on.

- ☐ Write your README covering all required sections: tool inventory (name, inputs with parameter names and types, outputs, purpose), planning loop explanation, state management approach, error handling per tool with at least one concrete example from your testing, and spec reflection.
- ☐ Add the AI usage section to your README. Describe at least 2 specific instances: what you gave the AI tool as input (which spec sections, what diagram), what it produced, and what you changed or overrode before using it.
- ☐ Record a 3–5 minute demo video showing: a complete multi-step interaction using all 3 tools with narration of what the agent is doing at each step, a moment where state passing between tools is visible or narrated, and at least one triggered failure with the agent's graceful response.

Checkpoint

Interface runs at `http://localhost:7860` and all three output panels populate correctly. README covers all required sections with substantive content. Demo is recorded and includes a complete interaction, visible state passing, and at least one error handling scenario.

Submitting Your Project

Submit all of the following through the Course Portal:

- ☐ Link to your forked GitHub repository
- ☐ `planning.md` in your repo root (written before implementation, updated before stretch features)

☐ `README.md` including:

- Tool inventory: name, inputs (parameter names and types, e.g. `description` (str)), outputs, and purpose of each tool (the documented inputs and return values must match your actual function signatures in the code)
- How the planning loop works (describe the conditional logic, not just "it decides what to do")
- State management approach: what is stored, when, and how it's passed between tools
- Error handling strategy for each tool, with at least one concrete example from your testing
- Spec reflection: one way the spec helped you, one way implementation diverged from it and why
- AI usage section: at least 2 specific instances describing what you directed the AI to do and what you revised or overrode

☐ Demo video (3–5 minutes) showing:

- A complete multi-step interaction from user query to fit card, using all 3 required tools
- Narration of what the agent is doing at each step (which tool is being called and why)
- State visibly or verbally passing between tools
- At least one triggered failure with the agent's graceful, informative response

How It's Graded

A detailed breakdown of graded features and points can be found on the course [grading](#) page.